动态规划与马尔可夫决策过程的算法分析

第六组 刘飞阳 许哲圣 唐铭浩

目录

CONTENTS

01 动态规划算法&马尔科夫链

02 马尔可夫决策过程（MDP）&贝尔曼方程

03 实际案例：金融交易场景

01

动态规划算法&马尔科夫链

动态规划的基本概念

- **动态规划的定义与应用场景**：动态规划是一种通过将复杂问题分解为重叠子问题，并利用子问题的解来高效求解原问题的算法设计方法。本质是将子问题的解存储起来，避免重复计算，从而提高效率。例如在最短路径问题中，动态规划可以高效地找到从起点到终点的最短路径。动态规划广泛应用于计算机科学、数学、经济学等领域。如在资源分配问题中，动态规划可以优化资源的分配方案，以达到最大效益。

- **动态规划的关键要素**：动态规划包含**阶段、状态、决策、策略**等关键要素。阶段表示问题的分解步骤，状态是每个阶段的特征描述，决策是每个阶段的选择，策略是整个过程的决策序列。例如在背包问题中，阶段可以是物品的选取顺序，状态是当前背包的容量，决策是选择或不选择某个物品，策略是所有物品的选择方案。

- **动态规划的主要特性**：动态规划具有**最优子结构和无后效性**。最优子结构是指问题的最优解包含其子问题的最优解。无后效性是指一旦当前状态确定，其后续决策不依赖于过去的状态。例如在最长递增子序列问题中，最优子结构体现在子序列的最优解是整个序列最优解的一部分，无后效性体现在当前元素的选择只与当前状态有关，与之前的路径无关。

马尔可夫链

① 数学定义

马尔可夫链是一种具有马尔可夫性质的随机过程。其数学定义为：$P(X_{t+1}=j|X_t=i,X_{t-1}=i_{t1},\ldots,X_0=i_0)=P(X_{t+1}=j|X_t=i)$，则该过程为马尔可夫链。

②数学性质

马尔可夫性是指未来状态的概率分布仅由当前状态决定，与历史路径无关。转移概率矩阵表示状态之间的转移概率。例如在股票价格预测问题中，状态空间可以是股票价格的不同区间，转移概率矩阵表示从一个价格区间转移到另一个价格区间的概率。

③应用案例

马尔可夫链在金融领域有广泛应用。例如在信用评级问题中，可以使用马尔可夫链预测企业的信用评级变化。在自然语言处理中，马尔可夫链可以用于语言模型的建模。例如在文本生成问题中，通过马尔可夫链可以生成符合语言规律的文本。

④举例

例如在天气预测问题中，假设明天下雨的概率只与今天是否下雨有关，而与过去的天气无关，这就是一个马尔可夫链。

02

马尔可夫决策过程（MDP）
&
贝尔曼方程

马尔可夫决策过程的组成要素

1 状态空间

系统所有可能状态的集合。例如，在一个机器人导航问题中，状态可以是机器人在地图上的不同位置。

2 动作空间

在每个状态下，可供选择的动作集合。比如机器人可以有前进、后退、左转、右转等动作。

3 转移概率

给定当前状态和动作，系统转移到下一个状态的概率分布。例如，机器人在当前位置执行前进动作后，到达下一个位置的概率。

4 奖励函数

当系统从一个状态转移到另一个状态时，给予的奖励值。奖励用于衡量不同决策序列的好坏，引导智能体做出最优决策。例如，机器人到达目标位置时会得到一个正奖励，撞到障碍物则会得到一个负奖励。

5 状态值函数

表示从特定状态出发，按照某种策略进行决策所能得到的回报期望值。例如在金融投资问题中，状态值函数可以表示在某个投资状态下，按照某种投资策略所能获得的期望收益。

贝尔曼方程

- **贝尔曼方程是状态值函数的递推关系。**它将状态值函数分解为即时奖励和长期奖励：即时奖励表示从s_t（当前状态）转换到s_{t+1}（下一状态）的奖励，即时奖励与状态转移相伴随，只要发生状态转移，就一定会产生对应的即时奖励；。长期奖励（也称回报）是与状态转移链相伴随，不同的状态转移链对应不同的长期奖励。

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}[R_{t+1}|S_t = s] + \gamma \mathbb{E}[G_{t+1}|S_t = s], \\
&= \underbrace{\sum_a \pi(a|s) \sum_r p(r|s,a) r}_{\text{mean of immediate rewards}} + \underbrace{\gamma \sum_a \pi(a|s) \sum_{s'} p(s'|s,a) v_\pi(s')}_{\text{mean of future rewards}}, \\
&= \sum_a \pi(a|s) \left[\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_\pi(s') \right], \quad \forall s \in \mathcal{S}.
\end{aligned}
$$

- 它表示从特定状态出发，按照某种策略进行决策所能得到的回报期望值。其中γ 表示为衰减系数（折扣因子），表示即时奖励在当前状态的折扣值。

贝尔曼方程的价值迭代算法和时间复杂度

贝尔曼方程是通过迭代更新价值函数来逐步逼近最优价值函数，并根据最优价值函数选择最优策略。

- **初始化价值函数**：对价值函数进行初始化。通常将价值函数的初始估计设为0或者随机值。
- **迭代更新价值函数**：将每个状态都通过贝尔曼最优方程进行价值函数的更新。贝尔曼最优方程表示了最优值函数之间的关系，用来更新值函数的估计。价值函数的公式更新为 $V_{k+1}(s) = \max_{s} \sum_{s'} p(s' \mid s,a)\,[r(s,a,s') + \gamma V_K(s')]$
- **策略改进**：在每次值函数更新后，根据新的价值函数来进行策略改进，即选择能够使价值函数最大化的动作作为最优策略。最优策略的选择是$\pi(s) = arg\max_{s} \sum_{s'} p(s' \mid s,a)\,[r(s,a,s') + \gamma V(s')]$
- 价值迭代算法保证在有限步内收敛到最优价值函数，由于价值函数每次更新都会使价值函数逼近最优价值函数，且在有限状态空间内，价值函数一定会收敛。最后，当价值函数不再发生变化时，即最优价值函数。

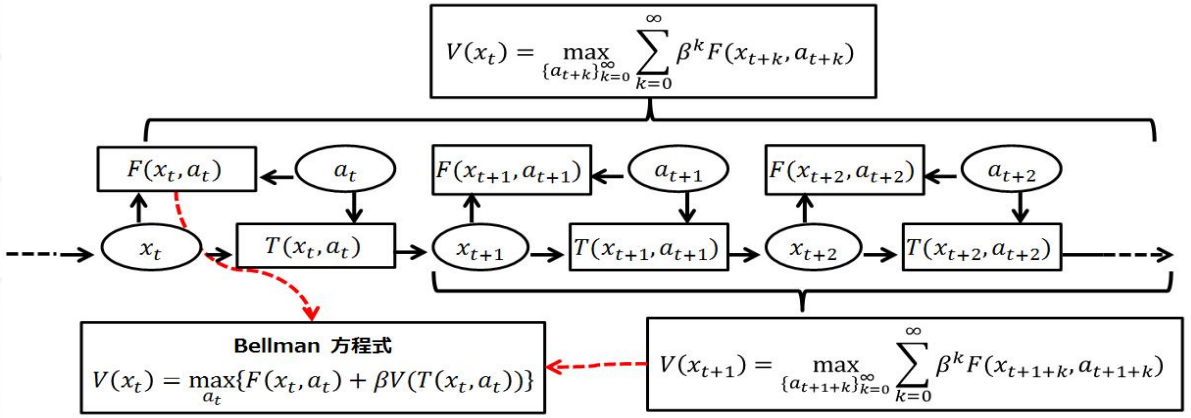
时间复杂度对比：矩阵形式推导（固定策略 π）：1）从贝尔曼期望方程出发：$V_\pi = R_\pi + \gamma P_\pi V_\pi$；2）转化：$(I - \gamma P_\pi) V_\pi = R_\pi$；3）闭式解：$V_\pi = (I - \gamma P_\pi)^{-1} R_\pi$；4）复杂度：$O(|S|^3)$。**价值迭代算法**：1）更新公式：$V_{k+1}(s) = \max_{s} \sum_{s'} p(s' \mid s,a)\,[r(s,a,s') + \gamma V_K(s')]$；2）单次迭代：$O(|S|^2|A|)$；3）总复杂度：$O(K \cdot |S|^2|A|)$。

03

实际案例：金融交易场景

例题：简单的金融交易场景

- **状态空间与动作空间**：状态 S = {s1，s2}，s1：上涨，s2：下跌；动作 A = {a1，a2}，a1：买入，a2：卖出
- **状态转移概率**：P(s1|s1,a1)=0.7，P(s2|s1,a1)=0.3，P(s1|s1,a2)=0.3，P(s2|s1,a2)=0.7，P(s1|s2,a1)=0.3，P(s2|s2,a1)=0.7，P(s1|s2,a2)=0.7，P(s2|s2,a2)=0.3
- **奖励函数与折扣因子**：R(s1,a1)=10，R(s1,a2)=−5，R(s2,a1)=−10，R(s2,a2)=5，折扣因子 γ = 0.9
- **当下策略 π**：在 s1 下：P(a1)=0.6，P(a2)=0.4；在 s2 下：P(a1)=0.3，P(a2)=0.7
- **最优策略与状态值函数**：通过贝尔曼方程可以得到各阶段状态价值函数，并通过迭代，可以得到各情况下的最优决策状态：在 s1 下选择 a1（买入），在 s2 下选择 a2（卖出）对应状态值：V(s1)=86.50，V(s2)=81.50。



Python代码分析

```python
import numpy as np

# 定义参数
gamma = 0.9  # 折扣因子
max_iterations = 100  # 最大迭代次数
epsilon = 1e-6  # 收敛阈值

# 奖励函数 R(s,a)
# (状态, 动作) -> 奖励值
# 状态: 0=上涨(s1), 1=下跌(s2)
# 动作: 0=买入(a1), 1=卖出(a2)
R = {
    (0, 0): 10,  # 上涨时买入
    (0, 1): -5,  # 上涨时卖出
    (1, 0): -10,  # 下跌时买入
    (1, 1): 5,  # 下跌时卖出
}

# 转移概率 P(s'|s,a)
# (状态, 动作) -> [转移到s1的概率, 转移到s2的概率]
P = {
    (0, 0): np.array([0.7, 0.3]),  # 上涨时买入 -> [0.7概率仍上涨, 0.3概率变下跌]
    (0, 1): np.array([0.3, 0.7]),  # 上涨时卖出 -> [0.3概率仍上涨, 0.7概率变下跌]
    (1, 0): np.array([0.3, 0.7]),  # 下跌时买入 -> [0.3概率变上涨, 0.7概率仍下跌]
    (1, 1): np.array([0.7, 0.3]),  # 下跌时卖出 -> [0.7概率变上涨, 0.3概率仍下跌]
}

# 初始化策略 π(a|s)
# pi[s, a] 表示在状态s下选择动作a的概率
pi = np.zeros((2, 2))
pi[0, 0] = 1.0  # 初始策略: 上涨状态(s1)选择买入(a1)
pi[1, 0] = 1.0  # 初始策略: 下跌状态(s2)选择买入(a1)

# 策略迭代主循环
iteration = 0
while iteration < max_iterations:
    iteration += 1
    print(f"\n第{iteration}轮策略迭代:")

    # 步骤1: 策略评估 - 计算当前策略下的状态值函数V(s)
    print("策略评估 - 计算当前策略下的值函数")
    V = np.zeros(2)  # 初始化值函数
    eval_iteration = 0

    while True:
        eval_iteration += 1
        V_new = np.zeros(2)

        for s in range(2):
            # 根据当前策略计算状态s的值
            for a in range(2):
                # V(s) += π(a|s) * [R(s,a) + γ * Σ P(s'|s,a) * V(s')]
                V_new[s] += pi[s, a] * (R[(s, a)] + gamma * np.dot(P[(s, a)], V))

        # 检查值函数是否收敛
        delta = np.max(np.abs(V_new - V))
        V = V_new  # 更新值函数

        if delta < epsilon:
            print(
                f"  值函数在{eval_iteration}次迭代后收敛: V(s1)={V[0]:.4f}, V(s2)={V[1]:.4f}"
            )
            break

        if eval_iteration > max_iterations:
            print("  值函数评估达到最大迭代次数")
            break

    # 步骤2: 策略改进 - 基于当前值函数V选择最优动作
    print("策略改进 - 更新策略")
    policy_stable = True
    new_pi = np.zeros((2, 2))

    for s in range(2):
        old_action = np.argmax(pi[s])

        # 计算每个动作的行动值函数Q(s,a)
        q_values = np.zeros(2)
        for a in range(2):
            # Q(s,a) = R(s,a) + γ * Σ P(s'|s,a) * V(s')
            q_values[a] = R[(s, a)] + gamma * np.dot(P[(s, a)], V)
```

Python代码分析

```python
        # 选择Q值最大的动作
        best_a = np.argmax(q_values)
        new_pi[s, best_a] = 1.0

        # 检查策略是否变化
        if old_action != best_a:
            policy_stable = False
            print(f"  状态s{s+1}的策略从a{old_action+1}变为a{best_a+1}")

    # 更新策略
    pi = new_pi

    # 如果策略稳定，结束迭代
    if policy_stable:
        print("策略已稳定，停止迭代")
        break

# 输出最终结果
print("\n最优策略: ")
for s in range(2):
    action_desc = "买入(a1)" if np.argmax(pi[s]) == 0 else "卖出(a2)"
    state_desc = "上涨(s1)" if s == 0 else "下跌(s2)"
    print(f"状态 {state_desc}: 动作 {action_desc}")

print(f"对应状态值函数: V(s1) = {V[0]:.4f}, V(s2) = {V[1]:.4f}")

# 计算最优策略下的Q值
print("\n最优策略下的行动值函数Q(s,a): ")
for s in range(2):
    for a in range(2):
        q_value = R[(s, a)] + gamma * np.dot(P[(s, a)], V)
        state_desc = "上涨(s1)" if s == 0 else "下跌(s2)"
        action_desc = "买入(a1)" if a == 0 else "卖出(a2)"
        print(f"Q({state_desc}, {action_desc}) = {q_value:.4f}")
```

```
第1轮策略迭代:
策略评估 - 计算当前策略下的值函数
  值函数在17次迭代后收敛: V(s1)=15.6250, V(s2)=-15.6250
策略改进 - 更新策略
  状态s2的策略从a1变为a2

第2轮策略迭代:
策略评估 - 计算当前策略下的值函数
  值函数评估达到最大迭代次数
策略改进 - 更新策略
策略已稳定，停止迭代

最优策略:
状态 上涨(s1): 动作 买入(a1)
状态 下跌(s2): 动作 卖出(a2)
对应状态值函数: V(s1) = 86.4980, V(s2) = 81.4980

最优策略下的行动值函数Q(s,a):
Q(上涨(s1), 买入(a1)) = 86.4982
Q(上涨(s1), 买入(a1)) = 86.4982
Q(上涨(s1), 卖出(a2)) = 69.6982
Q(下跌(s2), 买入(a1)) = 64.6982
Q(下跌(s2), 卖出(a2)) = 81.4982
Q(上涨(s1), 买入(a1)) = 86.4982
Q(上涨(s1), 卖出(a2)) = 69.6982
Q(下跌(s2), 买入(a1)) = 64.6982
Q(下跌(s2), 卖出(a2)) = 81.4982
Q(上涨(s1), 买入(a1)) = 86.4982
Q(上涨(s1), 卖出(a2)) = 69.6982
Q(上涨(s1), 买入(a1)) = 86.4982
Q(上涨(s1), 卖出(a2)) = 69.6982
Q(下跌(s2), 买入(a1)) = 64.6982
Q(下跌(s2), 卖出(a2)) = 81.4982
```

演示完毕 谢谢观看

汇报人：刘飞阳 许哲圣 唐铭浩

汇报时间：2025.4.30